

AAE 590LGM: Lie Group Methods
Problem Set 4

Travis Hastreiter

15 April, 2026

Contents

P1) Banana	3
P2) EKF	4
2.d)	4
2.e)	5
P3) ESKF	9
3.e)	9
3.f)	10
P4) LIEKF	15
4.d)	15
P5) Modelica/Rumoca	16

Code Listing

1	EKF Code	5
2	Kalman Filter Code	10
3	ESKF Code	11
4	Rumoca CodeGen	16

P1) Banana

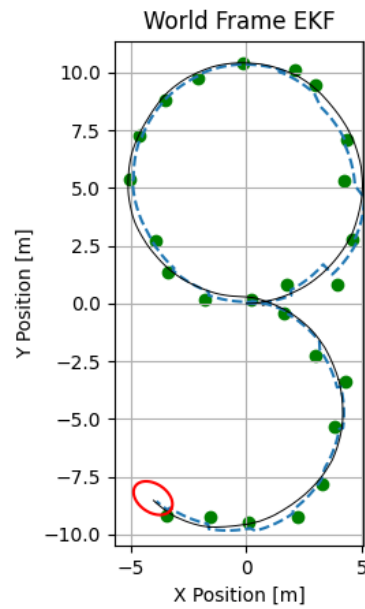
The Banana Distribution

P2) EKF

EKF in world frame

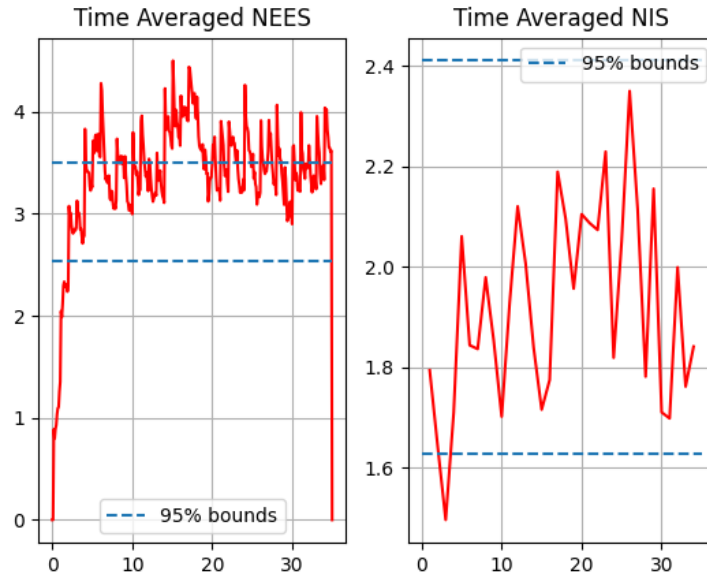
2.d)

Figure 1: Animation of EKF



2.e)

Figure 2: Monte Carlo Evaluation of EKF



Code Snippet 1: EKF Code

```

1 import numpy as np
2 import sympy as sp
3 import matplotlib.pyplot as plt
4 import matplotlib.animation as animation
5 from Code.SE2.SE2_maps import se2_compose, se2_exp, se2_log, se2_wedge, se2_vee, se2_inverse
6 from Code.SE2.SE2_integrators import se2_euler_integrator, se2_Lie_group_integrator
7 from Code.Plotting.covariance_plotting import plot_cov_ellipse, calc_and_plot_cov_ellipse,
8 from Code.Plotting.SE2_plotting import plot_scatter_unicycles,
9 from HW4.Parts.AAE590LGM_HW4_Problem import AAE590LGM_HW4_Problem
10 from Code.Filtering.KalmanFilter import EKF
11 from Code.Filtering.filter_performance_metrics import averaged_chi_squared_consistency_bands
12     , NEES
13 # Problem 2: World-Frame EKF
14
15 ## Unicycle SE2 Dynamics
16 #  $\dot{X} = X \hat{\xi}$ 
17 #  $\xi = [v; 0; \omega]$ 
18
19 ## Initialize Problem Parameters
20 # Simulation
21 prob = AAE590LGM_HW4_Problem()
22
23 # Monte Carlo
24 N_runs = 100
25
26 def EKF_SE2(prob):
27     t_ref = prob.t_k
28     X_ref = prob.simulate(np.zeros((3, prob.N_nodes)))
29

```

```

30 def f_disc(x_k, xi_k, dt):
31     x_kp1 = [x_k[0] + xi_k[0] * sp.cos(x_k[2]) * dt,
32             x_k[1] + xi_k[0] * sp.sin(x_k[2]) * dt,
33             x_k[2] + xi_k[2] * dt]
34     return x_kp1
35
36 # 2a) Prediction Step
37 x_sym = sp.symbols("x:3")
38 xi_sym = sp.symbols("xi:3")
39 f_disc_sym = sp.Matrix(f_disc(x_sym, xi_sym, prob.d_t))
40 F = f_disc_sym.jacobian(x_sym)
41 F_func = sp.lambdify((x_sym, xi_sym), F, "numpy")
42
43 f_disc_func = sp.lambdify((x_sym, xi_sym), f_disc_sym, "numpy")
44
45 L_k = lambda theta : prob.d_t * np.array([[np.cos(theta), 0, 0], [np.sin(theta), 0, 0], [0, 0, 1]])
46 Q_k_world = lambda x : L_k(x[2]) @ prob.Q_body @ L_k(x[2]).T
47
48 # 2b) Measurement Update
49 h_GPS = lambda x : x[0:2]
50 h_GPS_noise = lambda x : prob.rng.normal(h_GPS(x), prob.sigma_GPS)
51 R_z_GPS = prob.sigma_GPS ** 2 * np.eye(2)
52 H_GPS = np.hstack((np.eye(2), np.zeros((2, 1))))
53 H_GPS_func = lambda x : H_GPS
54
55 # 2c) Implementation
56 xhat_0 = np.zeros((3, 1))
57 P_0 = np.diag((0.01, 0.01, 0.01))
58
59 X_true = prob.monte_carlo(1)
60 x_true = np.zeros((3, prob.N_nodes))
61 for k in range(prob.N_nodes):
62     x_true[:, k] = se2_mat_to_tuplevec(X_true[:, :, k]).reshape((3,))
63
64 xhat_k = np.zeros((3, prob.N_nodes))
65 Xhat_k = np.zeros((3, 3, prob.N_nodes))
66 Phat_k = np.zeros((3, 3, prob.N_nodes))
67 xhat_k[:, 0] = xhat_0.reshape((3,))
68 Xhat_k[:, :, 0] = se2_tuplevec_to_mat(xhat_0).reshape((3, 3))
69 Phat_k[:, :, 0] = P_0
70 nu_k = []
71 K_k = []
72 innovation = []
73 t_k_GPS = []
74 z_k_GPS = []
75 k_GPS = []
76 for k in range(prob.N_steps):
77     if np.mod(t_ref[k], 1 / prob.GPS_rate) == 0 and k > 1:
78         z_GPS = h_GPS_noise(x_true[:, k])
79         z_k_GPS.append(z_GPS)
80         t_k_GPS.append(t_ref[k])
81         k_GPS.append(k)
82
83         xhat_k[:, k + 1], Phat_k[:, :, k + 1], nu_k_i, K_k_i, innovation_i = EKF(xhat_k[:, k], Phat_k[:, :, k], prob.xi_k[:, k], h_GPS, z_k_GPS, f_disc_func, F_func, Q_k_world, H_GPS_func, R_z_GPS)
84
85         nu_k.append(nu_k_i)
86         K_k.append(K_k_i)
87         innovation.append(innovation_i)
88     else:
89         z_GPS = np.nan
90
91         xhat_k[:, k + 1], Phat_k[:, :, k + 1] = EKF(xhat_k[:, k], Phat_k[:, :, k], prob.xi_k[:, k], h_GPS, z_k_GPS, f_disc_func, F_func, Q_k_world, H_GPS_func, R_z_GPS)
92         Xhat_k[:, :, k + 1] = se2_tuplevec_to_mat(xhat_k[:, k + 1]).reshape((3, 3))
93

```

```

94     nu_k = np.array(nu_k)
95     K_k = np.array(K_k)
96     innovation = np.array(innovation)
97     t_k_GPS = np.array(t_k_GPS)
98     z_k_GPS = np.array(z_k_GPS)
99     k_GPS = np.array(k_GPS)
100
101     return x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k, K_k, innovation, t_k_GPS, z_k_GPS,
        k_GPS
102 x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k, K_k, innovation, t_k_GPS, z_k_GPS, k_GPS =
        EKF_SE2(prob)
103
104 # 2d) Animation
105
106 fig, ax = plt.subplots()
107 ax.set_title("World Frame EKF")
108 ax.set_xlabel("X Position [m]")
109 ax.set_ylabel("Y Position [m]")
110 ax.grid()
111 plt.gca().set_aspect('equal', adjustable = 'box')
112 run_plot, run_quiver, ref_plot, ref_quiver = plot_scatter_unicycles_ax(X_true[:, :, 0],
        Xhat_k[:, :, 0].reshape((3, 3, 1, 1)), True, ax)
113 cov_plot = plot_cov_ellipse_ax(Phat_k[0:2, 0:2, 0], xhat_k[0:2, 0], "red", "-", ax)
114 gps_plot = ax.scatter(z_k_GPS[k_GPS <= prob.N_nodes, 0], z_k_GPS[k_GPS <= prob.N_nodes, 1],
        color = "green")
115
116 def update(k):
117     ref_plot[0].set_xdata(x_true[0, :k])
118     ref_plot[0].set_ydata(x_true[1, :k])
119     run_plot[0].set_xdata(xhat_k[0, :k])
120     run_plot[0].set_ydata(xhat_k[1, :k])
121     e_data = create_cov_ellipse(Phat_k[0:2, 0:2, k], xhat_k[0:2, k])
122     cov_plot[0].set_xdata(e_data[0, :] + xhat_k[0, k])
123     cov_plot[0].set_ydata(e_data[1, :] + xhat_k[1, k])
124     gps_plot.set_offsets(z_k_GPS[k_GPS <= k, :])
125     return (gps_plot, cov_plot[0], ref_plot, run_plot)
126
127 ani = animation.FuncAnimation(fig=fig, func=update, frames = prob.N_nodes, interval=10)
128
129 #plt.savefig("./HW4/Parts/Q1/AAE590LGM_HW4_Q2_ref.png")
130 plt.show()
131
132 # 2e) Monte Carlo Evaluation
133 nu_k_runs = np.zeros((N_runs, t_k_GPS.size))
134 eps_k_runs = np.zeros((N_runs, prob.N_nodes))
135 for i in range(N_runs):
136     x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k_runs[i, :], K_k, innovation, t_k_GPS,
        z_k_GPS, k_GPS = EKF_SE2(prob)
137
138     xhat_error = x_true[:, 0:] - xhat_k[:, 0:]
139     xhat_error[2] = np.mod(xhat_error[2] + np.pi, 2 * np.pi) - np.pi
140
141     eps_k = np.zeros((1, prob.N_nodes))
142     for k in range(2, prob.N_nodes - 1):
143         eps_k_runs[i, k] = NEES(xhat_error[:, k], Phat_k[:, :, k])
144
145     print(i)
146
147 nu_k_avg = np.average(nu_k_runs, 0)
148 eps_k_avg = np.average(eps_k_runs, 0);
149
150 eps_bounds = averaged_chi_squared_consistency_bands(N_runs, 3)
151 nu_bounds = averaged_chi_squared_consistency_bands(N_runs, 2)
152
153 plt.subplot(1, 2, 1)
154 plt.plot(prob.t_k, eps_k_avg, color = "red")
155 plt.title("Time Averaged NEES")
156 plt.hlines(eps_bounds, 0, prob.T, linestyles = "--", label="95% bounds")

```



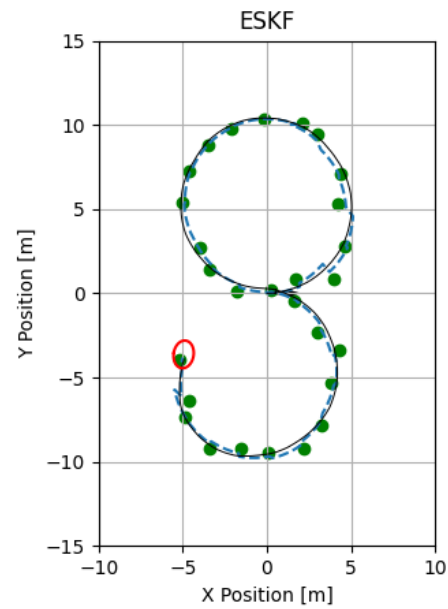
```
157 plt.grid()
158 plt.legend()
159
160 plt.subplot(1, 2, 2)
161 plt.plot(t_k_GPS, nu_k_avg, color = "red")
162 plt.title("Time Averaged NIS")
163 plt.hlines(nu_bounds, 0, prob.T, linestyle = "--", label="95% bounds")
164 plt.grid()
165 plt.legend()
166
167 plt.savefig("./HW4/Parts/Q2/AAE590LGM_HW4_Q2e_mc.png")
168
169 plt.show()
```

P3) ESKF

Error State EKF using Body Frame Error

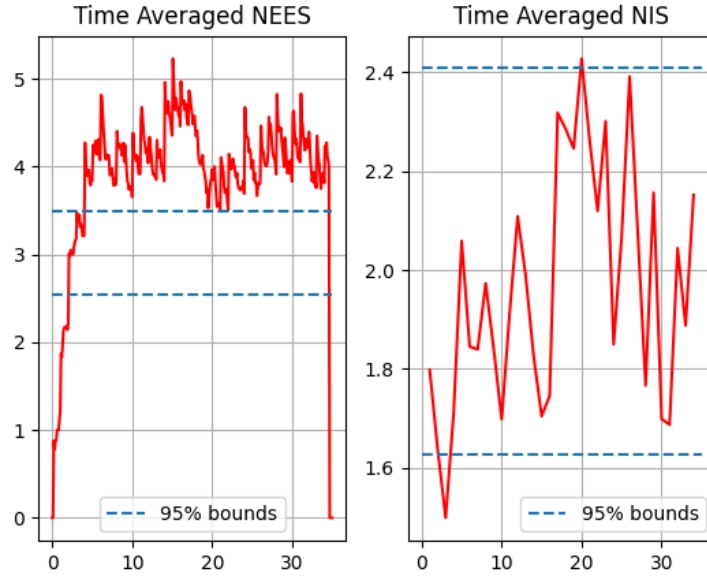
3.e)

Figure 3: Animation of ESKF



3.f)

Figure 4: Monte Carlo Evaluation of ESKF



Code Snippet 2: Kalman Filter Code

```

1 import numpy as np
2 from Code.Filtering.filter_performance_metrics import NIS
3
4 def discrete_covariance_prediction(P_k_k, F_k, Q_k):
5     P_kp1_k = F_k @ P_k_k @ F_k.T + Q_k
6     return P_kp1_k
7
8 def Kalman_gain(P_k_k, H, R_z):
9     S_k = H @ P_k_k @ H.T + R_z
10    S_k_inv = np.linalg.inv(S_k)
11    K_k = P_k_k @ (H.T @ S_k_inv)
12    return K_k, S_k_inv
13
14 def covariance_correction(P_k_k, K_k, H, R_z):
15     P_k_k = (np.eye(3) - K_k @ H) @ P_k_k @ (np.eye(3) - K_k @ H).T + K_k @ R_z @ K_k.T
16     return P_k_k
17
18 def EKF(x_k_k, P_k_k, xi_k, h, z_k, f_disc, F, Q_k, H, R_z):
19     n_x = x_k_k.size
20
21     # Predict
22     x_k_k = f_disc(x_k_k, xi_k)
23     P_k_k = discrete_covariance_prediction(P_k_k, F(x_k_k, xi_k), Q_k(x_k_k))
24
25     H_k = H(x_k_k)
26
27     # Correct if Measurement Exists
28     if not np.all(np.isnan(z_k)):
29         # Correct
30         K_k, S_k_inv = Kalman_gain(P_k_k, H_k, R_z)
31         zhat_k = h(x_k_k).reshape(z_k.shape) # Expected measurement
32         innovation = z_k - zhat_k
33

```

```

34     x_k_k = x_k_km1.reshape(n_x,) + K_k @ innovation
35     P_k_k = covariance_correction(P_k_km1, K_k, H_k, R_z)
36
37     # Filter performance metric (Normalized Innovation Squared - NIS)
38     nu_k = NIS(innovation, S_k_inv)
39     return x_k_k.reshape((n_x,)), P_k_k.reshape((n_x, n_x)), nu_k, K_k, innovation
40 else:
41     return x_k_km1.reshape((n_x,)), P_k_km1.reshape((n_x, n_x))
42
43 # def ESKF_update():
44 def ESKF(x_km1_km1, P_km1_km1, xi_k, h, z_k, f_disc, F, Q_k, H, R_z, x_update_func):
45     n_x = np.size(x_km1_km1, axis = 0)
46
47     # Predict
48     x_k_km1 = f_disc(x_km1_km1, xi_k)
49     P_k_km1 = discrete_covariance_prediction(P_km1_km1, F(x_km1_km1, xi_k), Q_k(x_km1_km1))
50
51     H_k = H(x_k_km1)
52
53     # Correct if Measurement Exists
54     if not np.all(np.isnan(z_k)):
55         # Correct
56         K_k, S_k_inv = Kalman_gain(P_k_km1, H_k, R_z)
57         zhat_k = h(x_k_km1).reshape(z_k.shape) # Expected measurement
58         innovation = z_k - zhat_k
59
60         delta_x = K_k @ innovation
61         x_k_k = x_update_func(x_k_km1.reshape(x_km1_km1.shape), delta_x)
62         P_k_k = covariance_correction(P_k_km1, K_k, H_k, R_z)
63
64         # Filter performance metric (Normalized Innovation Squared - NIS)
65         nu_k = NIS(innovation, S_k_inv)
66         return x_k_k.reshape(x_km1_km1.shape), P_k_k.reshape((n_x, n_x)), np.squeeze(nu_k),
        K_k, innovation
67     else:
68         return np.squeeze(x_k_km1.reshape(x_km1_km1.shape)), P_k_km1.reshape((n_x, n_x))
69
70 # def LIEKF_update():

```

Code Snippet 3: ESKF Code

```

1 import numpy as np
2 import sympy as sp
3 import matplotlib.pyplot as plt
4 import matplotlib.animation as animation
5 from Code.S02.S02_maps import so2_exp
6 from Code.SE2.SE2_maps import se2_compose, se2_exp, se2_log, se2_wedge, se2_vee, se2_inverse
7 from Code.SE2.SE2_integrators import se2_euler_integrator, se2_Lie_group_integrator
8 from Code.Plotting.covariance_plotting import plot_cov_ellipse, calc_and_plot_cov_ellipse,
9 from Code.Plotting.SE2_plotting import plot_scatter_unicycles,
10 from HW4.Parts.AAE590LGM_HW4_Problem import AAE590LGM_HW4_Problem
11 from Code.Filtering.KalmanFilter import ESKF
12 from Code.Filtering.filter_performance_metrics import averaged_chi_squared_consistency_bands
13     , NEES
14 # Problem 3: Error-State EKF in the BodyFrame
15
16 ## Unicycle SE2 Dynamics
17 # Xdot = X xi^hat
18 # xi = [v; 0; omega]
19
20 ## Initialize Problem Parameters
21 # Simulation
22 prob = AAE590LGM_HW4_Problem()
23

```

```

24 # Monte Carlo
25 N_runs = 100
26
27 def ESKF_SE2(prob):
28     t_ref = prob.t_k
29     X_ref = prob.simulate(np.zeros((3, prob.N_nodes)))
30
31     def f_disc(x_k, xi_k, dt):
32         x_kp1 = [x_k[0] + xi_k[0] * sp.cos(x_k[2]) * dt,
33                 x_k[1] + xi_k[0] * sp.sin(x_k[2]) * dt,
34                 x_k[2] + xi_k[2] * dt]
35         return x_kp1
36
37     def F_func(x_k, xi_k):
38         A = np.array([[1, xi_k[2] * prob.delta_t, 0],
39                       [-xi_k[2] * prob.delta_t, 1, xi_k[0] * prob.delta_t],
40                       [0, 0, 1]])
41         return A
42
43     def x_update_func(x_k_kml, delta_x_k):
44         x_k_k = np.zeros([3, 1])
45         x_k_k[0:2] = (x_k_kml[0:2] + so2_exp(x_k_kml[2]) @ delta_x_k[0:2]).reshape((2, 1))
46         x_k_k[2] = x_k_kml[2] + delta_x_k[2]
47
48         return x_k_k
49
50     def x_update_func_good(X_k_kml, delta_x_k):
51         X_k_k = se2_mat_to_tuplevec(se2_compose(se2_tuplevec_to_mat(X_k_kml), se2_exp(
52             delta_x_k)))
53         return X_k_k
54
55     # 2a) Prediction Step
56     x_sym = sp.symbols("x:3")
57     xi_sym = sp.symbols("xi:3")
58     f_disc_sym = sp.Matrix(f_disc(x_sym, xi_sym, prob.delta_t))
59
60     f_disc_func = sp.lambdify((x_sym, xi_sym), f_disc_sym, "numpy")
61     def f_disc_func(X_k, xi_k):
62         X_kp1 = se2_mat_to_tuplevec(se2_compose(se2_tuplevec_to_mat(X_k), se2_exp(xi_k *
63             prob.delta_t)))
64         return X_kp1
65
66     Q_k = lambda x : prob.delta_t ** 2 * prob.Q_body
67
68     # 2b) Measurement Update
69     h_GPS = lambda x : x[0:2]
70     h_GPS_noise = lambda x : prob.rng.normal(h_GPS(x), prob.sigma_GPS)
71     R_z_GPS = prob.sigma_GPS ** 2 * np.eye(2)
72     H_GPS_func = lambda x : np.hstack((so2_exp(x[2]), np.zeros((2, 1))))
73
74     # 2c) Implementation
75     xhat_0 = np.zeros((3, 1))
76     P_0 = np.diag((0.01, 0.01, 0.01))
77
78     X_true = prob.monte_carlo(1)
79     x_true = np.zeros((3, prob.N_nodes))
80     for k in range(prob.N_nodes):
81         x_true[:, k] = se2_mat_to_tuplevec(X_true[:, :, k]).reshape((3,))
82
83     xhat_k = np.zeros((3, prob.N_nodes))
84     Xhat_k = np.zeros((3, 3, prob.N_nodes))
85     Phat_k = np.zeros((3, 3, prob.N_nodes))
86     xhat_k[:, 0] = xhat_0.reshape((3,))
87     Xhat_k[:, :, 0] = se2_tuplevec_to_mat(xhat_0).reshape((3, 3))
88     Phat_k[:, :, 0] = P_0
89     nu_k = []
90     K_k = []
91     innovation = []

```

```

90     t_k_GPS = []
91     z_k_GPS = []
92     k_GPS = []
93     for k in range(prob.N_steps):
94         if np.mod(t_ref[k], 1 / prob.GPS_rate) == 0 and k > 1:
95             z_GPS = h_GPS_noise(x_true[:, k])
96             z_k_GPS.append(z_GPS)
97             t_k_GPS.append(t_ref[k])
98             k_GPS.append(k)
99
100         xhat_k[:, k + 1], Phat_k[:, :, k + 1], nu_k_i, K_k_i, innovation_i = ESKF(xhat_k
101        [:, k], Phat_k[:, :, k], prob.xi_k[:, k], h_GPS, z_GPS, f_disc_func, F_func, Q_k,
102         H_GPS_func, R_z_GPS, x_update_func)
103
104         nu_k.append(nu_k_i)
105         K_k.append(K_k_i)
106         innovation.append(innovation_i)
107     else:
108         z_GPS = np.nan
109
110         xhat_k[:, k + 1], Phat_k[:, :, k + 1] = ESKF(xhat_k[:, k], Phat_k[:, :, k], prob
111         .xi_k[:, k], h_GPS, z_GPS, f_disc_func, F_func, Q_k, H_GPS_func, R_z_GPS, x_update_func)
112         Xhat_k[:, :, k + 1] = se2_tuplevec_to_mat(xhat_k[:, k + 1]).reshape((3, 3))
113
114     nu_k = np.array(nu_k)
115     K_k = np.array(K_k)
116     innovation = np.array(innovation)
117     t_k_GPS = np.array(t_k_GPS)
118     z_k_GPS = np.array(z_k_GPS)
119     k_GPS = np.array(k_GPS)
120
121     return x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k, K_k, innovation, t_k_GPS, z_k_GPS,
122     k_GPS
123 x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k, K_k, innovation, t_k_GPS, z_k_GPS, k_GPS =
124     ESKF_SE2(prob)
125
126 # Rotate Phat_k into world frame
127 def rotate_covariance_to_world(Xhat_k, Phat_k):
128     Rh_k_mat = np.block([[Xhat_k[0:2, 0:2], np.zeros((2, 1))], [np.zeros((1, 2)), 1]])
129     Phat_k_world = Rh_k_mat @ Phat_k @ Rh_k_mat.T
130     return Phat_k_world
131
132 Phat_k_world = np.zeros_like(Phat_k)
133 for k in range(prob.N_steps):
134     Phat_k_world[:, :, k] = rotate_covariance_to_world(Xhat_k[:, :, k], Phat_k[:, :, k])
135
136 # 2d) Animation
137
138 fig, ax = plt.subplots()
139 ax.set_title("ESKF")
140 ax.set_xlabel("X Position [m]")
141 ax.set_ylabel("Y Position [m]")
142 ax.grid()
143 plt.gca().set_aspect('equal', adjustable = 'box')
144 run_plot, run_quiver, ref_plot, ref_quiver = plot_scatter_unicycles_ax(X_true[:, :, 0],
145     Xhat_k[:, :, 0].reshape((3, 3, 1, 1)), True, ax)
146 cov_plot = plot_cov_ellipse_ax(Phat_k_world[0:2, 0:2, 0], xhat_k[0:2, 0], "red", "-", ax)
147 gps_plot = ax.scatter(z_k_GPS[k_GPS <= prob.N_nodes, 0], z_k_GPS[k_GPS <= prob.N_nodes, 1],
148     color = "green")
149 ax.set_xlim([-10, 10])
150 ax.set_ylim([-15, 15])
151
152 def update(k):
153     ref_plot[0].set_xdata(x_true[0, :k])
154     ref_plot[0].set_ydata(x_true[1, :k])
155     run_plot[0].set_xdata(xhat_k[0, :k])
156     run_plot[0].set_ydata(xhat_k[1, :k])
157     e_data = create_cov_ellipse(Phat_k_world[0:2, 0:2, k], xhat_k[0:2, k])

```

```

151     cov_plot[0].set_xdata(e_data[0, :] + xhat_k[0, k])
152     cov_plot[0].set_ydata(e_data[1, :] + xhat_k[1, k])
153     gps_plot.set_offsets(z_k_GPS[k_GPS <= k, :])
154     return (gps_plot, cov_plot[0], ref_plot, run_plot)
155
156 ani = animation.FuncAnimation(fig=fig, func=update, frames = prob.N_nodes, interval=10)
157
158 plt.show()
159 #plt.savefig("../HW4/Parts/Q1/AAE590LGM_HW4_Q2_ref.png")
160
161
162 # 3f) Monte Carlo Evaluation
163 nu_k_runs = np.zeros((N_runs, t_k_GPS.size))
164 eps_k_runs = np.zeros((N_runs, prob.N_nodes))
165 for i in range(N_runs):
166     x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k_runs[i, :], K_k, innovation, t_k_GPS,
167     z_k_GPS, k_GPS = ESKF_SE2(prob)
168
169     Phat_k_world = np.zeros_like(Phat_k)
170     for k in range(prob.N_steps):
171         Phat_k_world[:, :, k] = rotate_covariance_to_world(Xhat_k[:, :, k], Phat_k[:, :, k])
172
173     xhat_error = x_true - xhat_k
174     xhat_error[2] = np.mod(xhat_error[2] + np.pi, 2 * np.pi) - np.pi
175
176     '''for k in range(2, prob.N_nodes - 3):
177         eps_k_runs[i, k] = NEES(xhat_error[:, k], Phat_k_world[:, :, k])'''
178     for k in range(2, prob.N_nodes - 4):
179         Xhat_error = se2_log(se2_compose(se2_inverse(Xhat_k[:, :, k]), X_true[:, :, k, 0]))
180         eps_k_runs[i, k] = NEES(Xhat_error, Phat_k[:, :, k])
181
182     print(i)
183
184 nu_k_avg = np.average(nu_k_runs, 0)
185 eps_k_avg = np.average(eps_k_runs, 0);
186
187 eps_bounds = averaged_chi_squared_consistency_bands(N_runs, 3)
188 nu_bounds = averaged_chi_squared_consistency_bands(N_runs, 2)
189
190 plt.subplot(1, 2, 1)
191 plt.plot(prob.t_k, eps_k_avg, color = "red")
192 plt.title("Time Averaged NEES")
193 plt.hlines(eps_bounds, 0, prob.T, linestyle = "--", label="95% bounds")
194 plt.grid()
195 plt.legend()
196
197 plt.subplot(1, 2, 2)
198 plt.plot(t_k_GPS, nu_k_avg, color = "red")
199 plt.title("Time Averaged NIS")
200 plt.hlines(nu_bounds, 0, prob.T, linestyle = "--", label="95% bounds")
201 plt.grid()
202 plt.legend()
203
204 plt.savefig("../HW4/Parts/Q3/AAE590LGM_HW4_Q3f_mc.png")
205 plt.show()

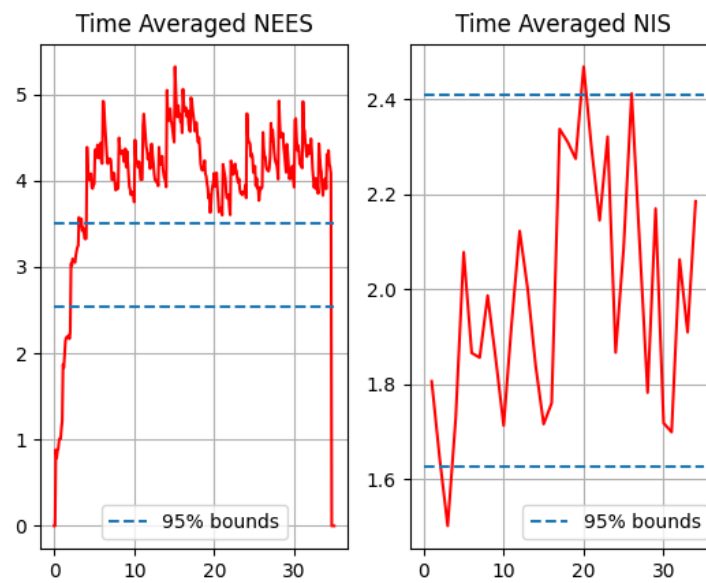
```

P4) LIEKF

Left Invariant Extended Kalman Filter

4.d)

Figure 5: Monte Carlo Evaluation of LIEKF



The LIEKF performs almost exactly the same as the ESKF which makes sense because the only difference is the exponential map.

P5) Modelica/Rumoca

Rumoca Time BABY

Figure 6: Rumoca



I did not have time to figure out how to hook up the generated code with a IVP solver. It seems like the SUNDIALS4py IVA solver would be ideal since Rumoca uses sundials to solve DAE equations. The trajectory here is more accurate than what I was using earlier because for my python code I was only sampling the twist every 0.1 seconds so it did not flip the angular velocity direction exactly at the right time, leading to an offset lower lobe. The Modelica trajectory is better because it solves the underlying differential equation instead of discretizing it.

Code Snippet 4: Rumoca CodeGen

```

1
2 """
3 Generated by Rumoca.
4
5 This template renders the current DAE context into a SymPy-friendly symbolic
6 model. It keeps Rumoca's continuous equations in residual form and can solve
7 for explicit state derivatives and algebraics when possible.
8 """
9
10 import sympy as sp
11
12
13 def der(symbol):
14     if isinstance(symbol, sp.Indexed):
15         return sp.IndexedBase(f"{symbol.base}_dot")[symbol.indices]
16     if isinstance(symbol, sp.IndexedBase):
17         return sp.IndexedBase(f"{symbol}_dot")
18     return sp.Symbol(f"{symbol}_dot")
19
20
21 def pre(symbol):
22     if isinstance(symbol, sp.Indexed):
23         return sp.IndexedBase(f"pre_{symbol.base}")[symbol.indices]
24     if isinstance(symbol, sp.IndexedBase):
25         return sp.IndexedBase(f"pre_{symbol}")
26     return sp.Symbol(f"pre_{symbol}")
27
28
29 def if_else(condition, then_expr, else_expr):
30     return sp.Piecewise((then_expr, condition), (else_expr, True))
31
32
33 def _vector(entries):
34     return (
35         sp.Matrix([symbol for _, symbol, _ in entries]),
36         {name: start for name, _, start in entries},
37         {name: index for index, (name, _, _) in enumerate(entries)},

```

```

38     )
39
40
41 def _column(entries):
42     return sp.Matrix(entries) if entries else sp.Matrix([])
43
44
45 def zeros(*dims):
46     if len(dims) == 1:
47         return sp.Matrix.zeros(int(dims[0]), 1)
48     return sp.Matrix.zeros(*(int(dim) for dim in dims))
49
50
51 def ones(*dims):
52     if len(dims) == 1:
53         return sp.Matrix.ones(int(dims[0]), 1)
54     return sp.Matrix.ones(*(int(dim) for dim in dims))
55
56
57 def sum(values):
58     if isinstance(values, sp.MatrixBase):
59         return sp.Add(*list(values))
60     if isinstance(values, (list, tuple)):
61         return sp.Add(*values)
62     return values
63
64
65 abs = sp.Abs
66 min = sp.Min
67 max = sp.Max
68 sign = sp.sign
69 sqrt = sp.sqrt
70 sin = sp.sin
71 cos = sp.cos
72 tan = sp.tan
73 asin = sp.asin
74 acos = sp.acos
75 atan = sp.atan
76 atan2 = sp.atan2
77 sinh = sp.sinh
78 cosh = sp.cosh
79 tanh = sp.tanh
80 exp = sp.exp
81 log = sp.log
82 log10 = lambda value: sp.log(value, 10)
83 floor = sp.floor
84 ceil = sp.ceiling
85 transpose = lambda value: sp.Matrix(value).T
86 identity = sp.eye
87 cross = lambda lhs, rhs: sp.Matrix(lhs).cross(sp.Matrix(rhs))
88 div = lambda lhs, rhs: sp.floor(lhs / rhs)
89 mod = sp.Mod
90 rem = sp.Mod
91
92
93 class Model:
94     """Flattened symbolic DAE model."""
95
96     def __init__(self):
97         self.time = sp.Symbol("time")
98         time = self.time
99
100         # States, algebraics, inputs, outputs, parameters, constants, discretetes.
101
102         theta = sp.Symbol("theta")

```

```

106     x = sp.Symbol("x")
107
108
109     y = sp.Symbol("y")
110
111
112     self.x, self.x_start, self.x_index = _vector([
113         ("theta", theta, 0),
114         ("x", x, 0),
115         ("y", y, 0)
116     ])
117
118
119
120     omega_adj = sp.Symbol("omega_adj")
121
122
123     tau = sp.Symbol("tau")
124
125
126     self.y, self.y_start, self.y_index = _vector([
127         ("omega_adj", omega_adj, 0),
128         ("tau", tau, 0)
129     ])
130
131
132     self.u, self.u_start, self.u_index = _vector([])
133
134
135     self.w, self.w_start, self.w_index = _vector([])
136
137
138
139     T = sp.Symbol("T")
140
141
142     T_fig8 = sp.Symbol("T_fig8")
143
144
145     T_lobe = sp.Symbol("T_lobe")
146
147
148     omega_turn = sp.Symbol("omega_turn")
149
150
151     v_nom = sp.Symbol("v_nom")
152
153
154     self.p, self.p_start, self.p_index = _vector([
155         ("T", T, 35),
156         ("T_fig8", T_fig8, (10 * pi)),
157         ("T_lobe", T_lobe, (5 * pi)),
158         ("omega_turn", omega_turn, 0.4),
159         ("v_nom", v_nom, 2)
160     ])
161
162
163
164     pi = sp.Symbol("pi")
165
166
167     self.constants, self.constants_start, self.constants_index = _vector([
168         ("pi", pi, 3.141592653589793)
169     ])
170
171
172     self.z, self.z_start, self.z_index = _vector([])
173

```

```

174
175     self.m, self.m_start, self.m_index = _vector([])
176
177
178     self.x_dot_alias, self.x_dot_alias_start, self.x_dot_alias_index = _vector([])
179
180     self.f_x = _column([
181         (der(x) - (cos(theta) * v_nom)),
182         (der(y) - (sin(theta) * v_nom)),
183         (tau - (time - (T_fig8 * floor((time / T_fig8))))),
184         (omega_adj - if_else((tau > T_lobe), (-omega_turn), omega_turn)),
185         (der(theta) - omega_adj)
186     ])
187
188     self.initial_residuals = _column([])
189
190     self.relations = _column([
191         (tau > T_lobe)
192     ])
193
194     self.explicit_targets = [
195
196         der(x),
197
198
199         der(y),
200
201
202         tau,
203
204
205         omega_adj,
206
207
208         der(theta),
209
210     ]
211     self.explicit_solution = None
212     self.explicit_rhs = sp.Matrix([])
213
214     def solve_explicit(self):
215         """Best-effort symbolic solve for explicit derivatives and algebraics."""
216         if len(self.f_x) == 0 or len(self.explicit_targets) == 0:
217             self.explicit_solution = {}
218             self.explicit_rhs = sp.Matrix([])
219             return self.explicit_solution
220
221         solutions = sp.solve(
222             list(self.f_x),
223             self.explicit_targets,
224             dict=True,
225             simplify=False,
226         )
227         self.explicit_solution = solutions[0] if solutions else {}
228         self.explicit_rhs = sp.Matrix(
229             [
230                 self.explicit_solution.get(symbol, symbol)
231                 for symbol in self.explicit_targets
232             ]
233         )
234         return self.explicit_solution
235
236     def summary(self):
237         return {
238             "states": [str(symbol) for symbol in self.x],
239             "algebraics": [str(symbol) for symbol in self.y],
240             "inputs": [str(symbol) for symbol in self.u],
241             "parameters": [str(symbol) for symbol in self.p],

```

```
242         "continuous_residual_count": len(self.f_x),
243         "initial_residual_count": len(self.initial_residuals),
244         "explicit_target_count": len(self.explicit_targets),
245     }
246
247
248 if __name__ == "__main__":
249     model = Model()
250     print(model.summary())
251     if len(model.f_x) > 0 and len(model.explicit_targets) > 0:
252         print(model.solve_explicit())
```